

Automated Software Testing - Exam Guidelines 2025-2026

These are a few primary guidelines for the exam. (The document also contains information about the 9 credit exam for the previous years.)

This document invalidates the guidelines of all the past years.

These are not meant to be exhaustive: we rely on common sense.

Remember that these guidelines will make sense only after you studied all the contents of this course.

Starting from this academic year, oral exams will take place exclusively on the scheduled exam-session dates.

It will therefore not be possible to arrange alternative or individualized dates.

The exam-session date corresponds to the oral exam date: students must attend on the date of the session they are registered for.

This change also applies to students from previous academic years, who are required to comply with this new procedure for oral exams.

The exam consists of

- a project to be developed at home (by a single student)
- an oral exam

The oral exam can be given only if the project is evaluated as sufficient, at least.

If the project is not sufficient, you must develop a new project entirely from scratch.

The oral exam must be given in the same exam session of the submitted project (see below for project submission).

If you don't give the oral exam in that session, the project will be invalidated, and you must develop a new project entirely from scratch.

The final grade will be the average of the project grade and the oral exam grade (if the oral exam is also sufficient).

The oral exam consists of questions on the project and on the topics of the course.

During the oral exam, the student must be able to describe and explain any part of the project (i.e., every single line of code or configuration file) and reproduce the build on their computer.

The students might also be asked to implement or modify something on the fly during the oral exam.

During the oral exam, the students must be able to provide formal and exhaustive answers to questions on any part of the course, that is, the book's contents. The students are expected to properly explain the definitions of concepts, tools and methodologies as described in the book and explained during the course.

If you fail the oral exam (or do not accept the proposed final grade), you must develop a new project entirely from scratch.

Each student must develop and submit an individual project. The project must be completed independently; group work, shared implementations, or joint development with other students are not allowed.

The student must first propose a project to the teacher by email. The teacher must approve the proposal (possibly requiring some modifications). Unless strictly necessary, please don't send proposals as attachments (Word documents, PDFs, etc.): just write the proposal in the body of the email.

I suggest you work on “a proof of concept implementation” of the final project before you propose it (this way, you have confidence in the feasibility of your proposal). A “Proof of Concept” (PoC) is a realization of a small and simple project to demonstrate its feasibility. A proof of concept is usually small and may or may not be complete. The PoC must not be proposed nor sent to the teacher. You do it to have confidence in the feasibility of your project before you propose it to the teacher.

IMPORTANT: Before you start thinking about a project proposal and working on the project, ensure that you have already studied and well comprehended all the course topics, including the examples.

The complexity of the main code is not essential (indeed, I suggest you implement a simple application). At the same time, the techniques taught during the course must be applied (the following list is not exhaustive, and it only contains the main mandatory parts):

- Test-Driven Development, unit tests, integration tests, and end-to-end tests
 - Mocking should be used appropriately (don't mock 3rd party types, see also the rest of the document)
 - The “shape” of the testing pyramid should be taken into account and respected
 - The strategies for testing will be crucial for the evaluation of the project
 - Tests must be readable and easily understandable (you can use additional testing libraries for this aim)
- Code Coverage (100% is required)
 - a part of the code can be excluded from code coverage, according to the suggestions explained in the book
 - Code Coverage report must be available both on Coveralls and SonarCloud
- Mutation Testing (“no survived mutant” is required)
 - Mutation Testing must be configured appropriately to avoid a tremendous amount of time for the build (focus on the essential parts of the code with business logic and apply mutation testing only there)
 - Some mutators can be excluded, but only by providing excellent motivations.
- Docker must be used in the project, especially for testing purposes when a server is needed (e.g., a database). Docker container(s) must be started automatically during the build, e.g., from Maven or using Testcontainers. DO NOT start containers from a GitHub Actions step. If, in order to run a test from Eclipse, a container must be manually started (because during the build it is started by the Maven plugin), it must be explicitly documented in the corresponding test class (including how the container must be started).
- Code Quality with SonarCloud (0 technical debt is required, i.e., no issue and no duplicated code, 100% code coverage also in SonarCloud is required)
 - Exclusions of rules and parts of the code are permissible if done appropriately and with excellent motivations.
- The code must be clean and neat, including excellent and consistent formatting and indentation of the code

- DO NOT mix spaces and tabs (use the command "Show Whitespace Characters" in Eclipse)
- Code modularity, which includes the structure of the project; e.g., in a Maven project, you might want to split the whole project into several submodules appropriately, if you think it makes sense.
- Build automation (with Maven)
- Continuous Integration (with GitHub Actions; in particular, focus on Linux in GitHub Actions; using Windows and macOS in GitHub Actions is optional).
- Git & GitHub
 - Although a single student works on the project, GitHub pull requests should be used for implementing single features
 - The teacher will inspect the Git history for the evaluation of the project
 - Make sure that no useless file (e.g., generated files) gets into the Git repository
- No *warning* must be present in any file of the project (see also the note about the Java version in the following)
 - warnings can be *suppressed* only with an excellent motivation

Things illustrated in the book and during the lessons should be starting points. However, we expect the students to be able to perform slight variations (e.g., in configurations) and to use more recent versions of dependencies and plugins.

However, the shape of the code, tests, configurations, etc., must not be too different from what has been taught in the course and written in the book and all the examples.

The implementation of a Java Swing graphical user interface is a strict requirement. The user interface must be thoroughly tested with the appropriate framework (unit, integration, and end-to-end tests).

You must use Java as the programming language.

Concerning Java, your project must be configured appropriately. In particular, it must be configured with the required version of Java. Use **ONLY** one of the following LTS versions of Java: 8, 11, 17, 21. No other Java versions are allowed.

*In the case of the **9 credits course**, you have to implement a web application. See the end of this document.*

The project must be accompanied by a brief report (about 10 pages) in PDF format, describing

- what you implemented
- applied techniques and frameworks
- design and implementation choices
- possible problems encountered and how you solved them
- descriptions of the development (and testing) of the most interesting parts

In the report, DO NOT define tools and mechanisms described in the course (e.g., do not explain what TDD is, etc.)

Italian students are allowed to write the report in Italian. They are also allowed to do the oral exam in Italian.

The complete build and test of the project must also be reproducible easily on the local machine (of the teacher), and the instructions to achieve that must be part of the report. The instructions to run the build must be straightforward and very short (e.g., a single Maven command, assuming that Java 8, 11, 17, and 21 are already installed on the teacher's machine; you can assume that Docker and Docker compose are already installed on the teacher's machine).

Project submission

The project must be available as a public GitHub repository.

The report can be part of the GitHub repository, but it is not strictly required.

On the first page of the PDF report, provide a link to the GitHub repository.

When an exam session's registration opens, a dedicated submission section will be available on the Moodle website (at the end of the page).

The submission will be open for **4** days. ("project submission window")

The students must use this section on Moodle to submit only the PDF report (remember, on the first page, you must provide the link to the GitHub repository).

Once submitted, the submission cannot be modified.

Changes made to the GitHub repository after the submission will not be considered.

Thus, ensure that everything is correct and working, and all requirements are fulfilled before the final delivery.

If you want to cancel your project submission, write an email to the teacher (but no further submission will be allowed for that exam session).

A project must be cancelled within the 4 days of the project submission window.

I will provide the results of the evaluations directly on Moodle, a few days before the oral exam date.

BEFORE submitting the project, register for that exam session.

Submitted projects without a corresponding exam session registration (within the 4 days of the project submission window) will not be evaluated in that exam session.

Even if you are not 100% sure you submit the project in an exam session, register to that session anyway. If you are not able to submit the project, you can always cancel your registration.

Students who attended the course in the past years, when the course had a different name ("Advanced Programming Techniques", "Advanced Techniques and Tools for Software Development", etc.) should have already registered for a past exam session (as prescribed in the past years' exam guidelines).

If they had never registered, and they are not able to register for an exam session for "Automated Software Testing", they should send me an e-mail so that I can manually register them for a past session, specifying name, surname, student id, name of the course they attended, id of the course, academic year of the course.

Hints for the projects with the corresponding evaluation/rating metrics

The final grade of the exam highly depends on the proposed project (low-rating, high-rating), besides, of course, the correctness and cleanness of the project's development (according to the topics of this course).

Note that a low-rating project could still be enough to pass the exam, but, in any case, the final grade for a low-rating project might not be that high and will never be the highest grade. On the other hand, proposing a high-rating project does not necessarily imply a high grade, but it is a precondition if you aim for the highest grade.

Below are a few suggestions for determining whether a proposed project is considered a low or a high rating. These are only a few suggestions.

Low rating:

- A variation of the example application seen in the course, without any new implementation/technology/tool/framework

High rating:

- A variation of some of the technologies seen in the course or the use of another technology (this requires learning a slightly different and new technology):
 - use another database instead of MongoDB, or configure your application to use different databases (e.g., MongoDB and MySQL), making it easy to specify which database to use
 - use advanced features of MongoDB (e.g., relations and transactions)
 - use a SQL database and the corresponding Java technologies (e.g., JPA, Hibernate, etc., with relations and transactions)
 - use Google Guice, a dependency injection framework, (see the last chapter of the book) to make an application easily configurable at runtime
 - write an application with both a GUI and a command-line UI
 - use BDD with Cucumber (see the chapter "End-to-End tests" in the book)
 - use another framework for BDD (e.g., JBehave, Jgiven, etc.)

If you use a new technology, in the report you should describe in detail how you used it. For example, if you use MongoDB transactions, you should describe how you used it (including a brief explanation on how they are meant to work in MongoDB).

If you implement transactions (with any database), make sure to follow the suggestions about transactions in the book. Other implementations will not be considered (and the project will likely be not sufficient).

The domain model of the application must be kept simple.

An example of an excellent domain model is

- "Todo" with a description and a boolean field (e.g., done)
 - "Todo" has a many-to-many relation with "Tag."
- Instead of "Todo" and "Tag," "Student" and "Teacher," or something similar
- 2 entities are enough for the project; in general, try to use at most 3 entities
- Some kind of relations between the entities should be present, relying on the mechanisms provided by the database

When proposing the project, in the e-mail, you must make explicit whether you plan to implement a low-rating or a high-rating project. In case you plan to propose a high-rating project, you have to state what should make it a high-rating project. The same holds when you deliver the final project.

TIP: if you plan to use a new technology, we suggest you use *learning tests* (see the book's final chapter), which can be put in the final project as well.

IMPORTANT

The following points are necessary for the project to be evaluated.

If a project is not evaluated, it can be re-submitted after properly fixing it in one of the future exam sessions.

- The project must be made available as an Eclipse project (or several Eclipse projects if you use multi modules). Do not use any other IDE.
- The Eclipse project(s) must be properly configured with one of the admitted Java versions.
- Ensure all the Eclipse projects' metadata files are in the Git repository (e.g., .project, .classpath, .settings, etc.).
- Ensure all the tests can be run from Eclipse with "Run As -> JUnit Test".

- Do not put generated files (e.g., target folder, .class files) or any other unrelated files (e.g., .DS_Store files) in the Git repository.
- Do not use frameworks like Lombok or model mappers.
- Never put a token or a password as clear text in the Git repository! Use the techniques provided by the used tools and services to encrypt such sensitive data safely.
- The main README.md file must contain the badges for the build status, the Coveralls code coverage and the SonarCloud badges.
- 100% code coverage on Coveralls and SonarCloud (as said at the beginning, some parts can be excluded from code coverage, but the final coverage must be 100%)
- 0 surviving mutants (as said at the beginning, the scope of mutation testing should be appropriately configured)
- 0 issues, 0 duplicated code on SonarCloud (as said at the beginning, proper rule exclusion is admissible if properly motivated)

The teacher will clone the Git repository and import the contents as Eclipse project(s). Once imported, the projects must not contain any errors or warnings. Please, make sure that this is the case (e.g., before submitting the final project, try to clone your Git repository from scratch and import it into a fresh Eclipse workspace where you configured Java 8, or 11, or 17, or 21, depending on the Java version you used).

The following points are necessary for the project to be at least sufficient.

Remember that 3rd party code MUST NEVER BE MOCKED.

Mocking types that “you don’t own” will make the project not sufficient.

Examples of violations that will make the project NOT sufficient are: mocking the EntityManager for JPA or the MongoClient for MongoDB, stubbing their methods and/or verifying their methods; this includes spying, of course, which is a form of mocking.

If you cannot reach 100% code coverage without mocking 3rd party code, then you have a problem in your code (useless conditions) or in your tests, or both.

In general, violating any best practice taught in the course or making typical mistakes that have been taught NOT to make will make the project NOT sufficient.

DON’T rely on material you find online, which will not necessarily be considered authoritative. When in doubt, ask the teacher. In general, only refer to the contents of the course (book and examples).

What you find online must never replace the topics as seen in the course.

If you use some documentation on-line for new mechanisms or using new libraries/frameworks/tools not seen in the course, ensure you cite the on-line sources you used in the report.

Ensure that documentation is authoritative; when in doubt, ask the teacher.

Submit your project ONLY after you are 100% sure you followed ALL the guidelines mentioned in this document.

You cannot ask for a pre-evaluation: the project will be evaluated ONLY after the final delivery.

Projects and implementations must not be similar to those of other students, so it is recommended NOT to work on the project together with other colleagues and NOT to compare yourselves with each other about the project in any way. Projects deemed similar (even if compared to projects submitted in the past) will be insufficient.

DO NOT search online for possible solutions or inspirations: they will make your code similar to existing projects.

Remember that if you apply a suggestion found online (or any “hack”) to fix a problem you encountered, you must then be able to explain all the details of that solution.

DO NOT use any tools related to artificial intelligence.

Even in the case of suspected copying or usage of AI tools, the project will be NOT sufficient. The reasons that lead to suspected copying or usage of AI tools, in addition to what is described above, include inconsistent code formatting and inconsistent use of TABs and spaces, and not being able to comment and/or explain parts or details of the project during the oral exam.

9 credits exam

IGNORE THIS SECTION if you attend the 6 credit course.

Once again, remember that these guidelines will make sense to you only after you study the contents of this course.

What is described in this document up to this point still holds for the 9-credit exam (including the low-rating and high-level rating concepts). However, for the 9 credits exam, you must implement a web application instead of a desktop one.

You must implement a web user interface (also tested): you cannot simply implement a REST server. End-to-end tests are also a strict requirement.

For the domain model of the web application, refer to what has already been said in the previous parts of this document.

Here are some suggestions for the high-level rating.

High rating:

- Use Spring Boot with one of the following variations:
 - use several databases in the same application, making it easy to switch among database implementations (e.g., for Spring Boot, you might use Spring Boot Profiles or other configuration mechanisms)
 - use BDD
 - use another framework for BDD (e.g., JBehave, Jgiven, etc.)
- Additional testing frameworks that could be used are
 - Wiremock (for mocking the server)
 - Pact (for contract testing, e.g., when implementing both a REST server and a REST client)